

Figure 1: Hash values on a circular dial

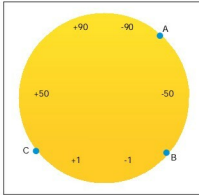


Figure 2: Memcached nodes on the circular dial along with hashes

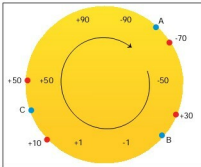


Figure 3: Nodes, Hash of keys on circular dial with Hashes

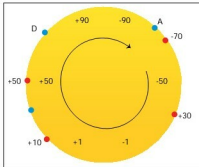


Figure 4: Removing a node

ensure that there is little collision). So if I were to map it to an instance, I would use the following command:

```
12356 % 18 = 6
```

This means that the data for the key 'Hello' would be stored in the instance number 6.

Now let us add a couple of instances, taking the count of instances to 12. Where would the key 'Hello' map to now?

```
12356 % 12 = 8
```

Because the client will look for the key 'Hello' in the eighth instance, it will no longer be able to find the value. This is why we use consistent hashing.

So what is consistent hashing? Simply put, it is a way of ensuring that keys map consistently to the same cache instance even when the cache instances are added or removed. The caching function does its best to make this scenario possible. But there will be some cache misses.

How does consistent hashing achieve that? Simple! It hashes the identifier for the caches (typically IP addresses and port combinations) with the same hashing function used to hash the key, and then applies a clever trick to map the keys to the instances.

Assume that the hashing function can only create hashes in the range -100 to +100 (it would be a pretty useless hashing function if it had only 201 possible values, but for the sake of demonstration, let us work with it). Now assume that the hash values were the dial of a clock (arranged in a circle just like they are on a clock). So the values would start at -100 at the top and increase clockwise until they reach +100 at full circle (see Figure 1).

Adding and removing instances

Now, let's hash the instances and plot the resulting hash (which will be in the range -100 to +100) on the dial. Let us assume the instances are at points A, B and C as shown in Figure 2.

Also, let the keys hash to the values -70, -30, 10 and 50 (as shown in Figure 3).

Now, to map keys to an instance, move clockwise and assign each key to the nearest instance that comes after the key. So, in this case, -70 and -30 will go to B, +10 will go to C and +50 will go to A.

What happens if an instance is removed? Let us assume that the instance B is removed. Then the values -70, -30 and +10 will go to C and the others will remain as is. Even after removing an instance, only two keys are re-mapped. The others will continue to be served from the same cache instance.

Now let us add another instance (see Figure 4). Say we added D at the location shown in the diagram. What would happen is that -70, -30 and +10 will still map to C, +50 would map to D, and A would have nothing mapped to it. Again, you will see that the cache has not been disrupted too much in this case.

Consistent hashing is now included in most of the popular Memcached clients. For example, Memcached Java Client, a popular Java client for Memcached, has support for it. 

References

- [1] http://weblogs.java.net/blog/tomwhite/archive/2007/11/consistent_hash.html
- [2] <http://www8.org/w8-papers/2a-webserver/caching/paper2.html>

By: Harish Babu

The author, employed by a leading telecom VAS provider, is an open source fan who likes exploring new technologies. He can be reached at [harish\(dot\)babu\(at\)gmail\(dot\)com](mailto:harish(dot)babu(at)gmail(dot)com).